DVR Optimization Problem

Jason Joy Manoj

July 19, 2026

1 Notation

- v: the verifier assigned to check the draft model's output. Takes one of four values: none, small, medium, large.
- k: the draft length, the number of tokens the draft model proposes before verification. Takes one of three values: 3, 5, or 7. Only applies when $v \neq$ none.
- (v, k): a candidate, one specific combination of verifier and draft length. There are 10 candidates total: none (1), plus small, medium, large each at $k \in \{3, 5, 7\}$ (9).
- query: the text of the incoming prompt.
- f_{enc}: a frozen, pretrained sentence encoder (all-mpnet-base-v2) that converts text into a fixed-length vector.
- $g(v, k)$: the candidate's embedding, built from a written description of verifier v passed through the same encoder, with the numeric value of k concatenated alongside it.
- $\oplus$: concatenation of two vectors into one longer vector.
- $z(v, k) = f_{\text{enc}}(\text{query}) \oplus g(v, k)$: the combined input vector fed to the router MLP.
- $\sigma(x) = \dfrac{1}{1 + e^{-x}}$: the sigmoid function.
- $A(v, k)$: the router's predicted accuracy for candidate (v, k), given the current query. Computed live, once per query, for all 10 candidates.
- $E(v, k)$: the raw measured energy cost (grams of CO_2) of candidate (v, k), measured offline via CodeCarbon.
- $T_{\text{raw}}(v, k)$: the raw measured latency (verification time plus cross-node transmission time) of candidate (v, k), measured offline.
- $\text{EnergyNorm}(v, k)$, $\text{TimeNorm}(v, k)$: $E(v, k)$ and $T_{\text{raw}}(v, k)$ rescaled to $[0, 1]$ via min-max normalization across the 10 candidates. Higher value means worse (more carbon, slower).
- w_A, w_E, w_T: independent weights, each in $[0, 1]$, summing to 1, controlling how much Accuracy, Energy, and Time count toward the final decision.
- $\text{CombinedScore}(v, k)$: the final score computed for each candidate.
- (v^*, k^*): the winning candidate, the verifier and draft length used to answer the query.

2 Accuracy Term

For each of the 10 candidates (v, k), the predicted accuracy $A(v, k)$ is produced by a four-step calculation:

1. The query is encoded into a vector by f_{enc}. Separately, a written description of verifier v is encoded by the same encoder, and the numeric value of k is attached alongside it. These are concatenated into $z(v, k)$.
2. $z(v, k)$ is passed through the router MLP, producing one raw scalar.
3. The raw scalar is passed through σ, squashing it into $[0, 1]$.
4. The result is $A(v, k)$, the predicted accuracy for that candidate.

$$A(v, k) = \sigma\big(\text{MLP}(z(v, k))\big) \tag{1}$$

Description: $A(v, k)$ is the router's live, per-query estimate of how likely candidate (v, k) is to answer this specific query correctly. It is the only one of the three objective terms that is computed on the fly for every incoming query; Energy and Time are looked up from a fixed offline table (Section 3). This is repeated for all 10 candidates, producing 10 accuracy values per query. Training labels for $A(v, k)$ are gathered offline: the pipeline is run on datasets with known correct answers, trying different (v, k) combinations, and each run's output is scored against the gold-standard answer to produce a label.

3 Energy and Time Terms

Unlike accuracy, the Energy and Time terms do not require a live predictor. Both are known, fixed values per candidate, measured once offline and reused for every query.

$$\text{EnergyNorm}(v, k) = \frac{E(v, k) - \min_{v', k'} E(v', k')}{\max_{v', k'} E(v', k') - \min_{v', k'} E(v', k')} \tag{2}$$

Description: Rescales the raw measured CO_2 cost of each candidate onto $[0, 1]$, relative to the cheapest and most expensive candidates in the pool. The candidate with the lowest measured energy gets EnergyNorm = 0; the most expensive gets EnergyNorm = 1. This is a min-max normalization, computed once offline from the profiling table and reused unchanged for every query.

$$\text{TimeNorm}(v, k) = \frac{T_{\text{raw}}(v, k) - \min_{v', k'} T_{\text{raw}}(v', k')}{\max_{v', k'} T_{\text{raw}}(v', k') - \min_{v', k'} T_{\text{raw}}(v', k')} \tag{3}$$

Description: Same min-max normalization, applied to raw measured latency instead of energy. Both Eq. 2 and Eq. 3 produce values in $[0, 1]$ where higher means worse (more carbon-costly, or slower) — oriented so that both are subtracted, not added, in the final score.

4 Combined Optimization Problem

We want to, for a given query:

$$\begin{aligned} \text{maximize} \quad & A(v, k) \\ \text{minimize} \quad & \text{EnergyNorm}(v, k) \\ \text{minimize} \quad & \text{TimeNorm}(v, k) \end{aligned}$$

These three goals are combined into a single score per candidate, with each objective given its own independent weight:

$$\text{CombinedScore}(v,k) = w_A \cdot A(v,k) \;-\; w_E \cdot \text{EnergyNorm}(v,k) \;-\; w_T \cdot \text{TimeNorm}(v,k) \tag{4}$$

Description: A weighted linear combination (scalarization) of the three objectives into a single number per candidate. Accuracy is added because higher is better; Energy and Time are subtracted because higher is worse. The three weights are independent of one another, so each objective's importance can be tuned separately during ablations (e.g. an energy-first configuration raises w_E without changing w_T).

The optimization problem is:

$$(v^*, k^*) = \arg\max_{(v,k)} \text{CombinedScore}(v,k) \tag{5}$$

$$\begin{aligned} \text{subject to:} \quad & v \in \{\text{none}, \text{small}, \text{medium}, \text{large}\} \\ & k \in \{3,5,7\} \text{ if } v \neq \text{none} \\ & w_A + w_E + w_T = 1, \quad w_A, w_E, w_T \geq 0 \end{aligned}$$

Description: Out of the 10 candidates, the router selects the (v,k) pair with the highest CombinedScore; this becomes the verifier and draft length used for the query. The weight-sum constraint is not required for the arg max itself to function (argmax is invariant to uniform rescaling) — it is imposed so that w_A, w_E, w_T remain interpretable as a trade-off allocation across the three objectives when swept during ablations.

5 Background: Why Latency Varies with k and Verifier Size

$T_{\text{raw}}(v,k)$ is measured directly rather than derived analytically, but the reason latency varies across candidates is well captured by the closed-form speedup expression from speculative decoding theory [1]. Given an acceptance rate α, a draft length γ, and a cost ratio c (time for one draft-model pass divided by time for one verifier-model pass), the expected speedup over standard autoregressive decoding is:

$$\text{Speedup}(\alpha,\gamma,c) = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)} \tag{6}$$

Description: A higher acceptance rate increases expected speedup, since more drafted tokens survive verification per call. A larger cost ratio (verifier slow relative to the drafter) reduces speedup, since $(\gamma c+1)$ grows with $\gamma = k$. This motivates why larger verifiers and larger k trade accuracy against latency in opposite directions — but it is not used directly inside CombinedScore, since $T_{\text{raw}}(v,k)$ is taken from offline profiling rather than computed live per query.

- **Eq. 1 (Accuracy term structure). Sourced.** Directly adapted from RTR's performance predictor, $\hat{a}_{i,j,k} = \sigma(\text{MLP}_{\text{perf}}(z_{i,j,k}))$ (RTR, WWW 2026, Eq. 4). Same structure, renamed to match DVR's notation.
- **Eqs. 2–3 (Min-max normalization). Not sourced from a specific paper.** A standard, general-purpose normalization technique used broadly in statistics and ML; not tied to any single publication. Adopted here as the author's own design choice to give Energy and Time comparable $[0,1]$ scales, replacing an earlier undefined normalization step.
- **Eq. 4 (Weighted three-term CombinedScore). Not sourced from a specific paper.** A standard linear scalarization approach from multi-objective optimization. Structurally similar in spirit to RTR's own two-term, single-weight score (RTR, Eq. 9), extended here to three independently-weighted terms. This extension is the author's own formulation, not copied from a published equation.
- **Eq. 6 (Speedup formula). Sourced.** Verified directly against Leviathan, Kalman & Matias, *Fast Inference from Transformers via Speculative Decoding*, ICML 2023 (arXiv:2211.17192) [1]. Formula and definition of the cost ratio c both confirmed to match the original.

Summary: 2 of the 4 core equations are adapted directly from cited papers (Eq. 1 from RTR, Eq. 6 from Leviathan et al.); 2 are standard techniques applied as the author's own design choices (Eqs. 2–3, 4), not attributable to a single source.

References

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. *Fast Inference from Transformers via Speculative Decoding*. Proceedings of the 40th International Conference on Machine Learning, PMLR 202:19274–19286, 2023. arXiv:2211.17192.